Contents

1 First-Order Logics 1
1.1 Syntax and Semantics of first-order logic 1
1.1.1 Syntax 1
1.1.2 Tensor Representation of Semantics 1
1.1.3 Worlds 2
1.1.4 Two Tensor Structures 2
1.2 Basis calculus for substitution structure 3
1.3 Substitution Structure 3
1.3.1 Grounding Tensors 3
1.3.2 Basis encodings of terms 3
1.3.3 Atomic sentences 4
1.3.4 Complex sentences 4
1.3.5 Formula Synthesis by Connectives 5
1.3.6 Quantifiers 5
1.3.7 Storage in Basis CP Decomposition 7
1.3.8 Example: Relational Databases 7
1.4 Knowledge bases 8
1.4.1 Substitution 8
1.5 Inference 8
1.5.1 Unification 9
1.5.2 Modus ponens 9

Chapter 1
First-Order Logics

Abstract To introduce the syntax and semantics of first-order logics, we in this chapter introduce the substitution tensor structure. This allows to capture structured representations of systems, beyond the factored representations introduced in the first two chapters.

1.1 Syntax and Semantics of first-order logic

The framework of first-order logic generalizes propositional logic analogously to the generalization of factored system representations towards structured system representations [**?**]. This more expressive framework is designed to reason about relations and functions between objects. To capture this framework by tensors we here first define the syntax and then investigate tensor representations of the semantics.

1.1.1 Syntax

A first-order logic theory consists in a finite set of constant symbols $\mathcal{U}$, a set of function symbols $\{v_l : l \in [r]\}$, a set of predicate symbols $\{g_k : k \in [d]\}$ and an arity map assigning the arity n_l to the function v_l and the arity n_k to the predicate g_k.

1.1.2 Tensor Representation of Semantics

We here follow the model-theoretic semantics of first-order logic, where sets of possible worlds are considered. We only treat in this work database semantics, where we assume that the domain $\mathcal{U}$ of each world has a one-to-one mapping to the constants of the theory and is therefore constant among the worlds. Database semantics is central to combine the semantic structure with the substitution structure. Under this assumption, the dimension of the substitution structure does not depend on the specific world, and we can combine both structures in a single tensor space to be defined in the next sections.

To each world there is a world domain $\mathcal{U}$ of objects, which we assume to be finite. We exploit the set-encoding formalism discussed in more detail in Chapter **??** and use bijective index interpretation maps

$$I : [r] \to \mathcal{U} .$$

A so-called term variable O takes states $o \in [r]$, which represent objects

$$I(o) \in \mathcal{U} .$$

The relations between objects are described by n-ary predicates g. Given a specific world x_W the truth of relations is represented by boolean tensors

$$g|_{x_W} \; : \; \bigtimes_{l \in [n]} [r] \to \{0, 1\} \, .$$

Given a tuple $o_0, \ldots, o_{n-1} \in \bigtimes_{l \in [n]} [r]$ the boolean

$$g|_{x_W} \, [O_0 = o_0, \ldots, O_{n-1} = o_{n-1}] \in \{0, 1\}$$

is called a grounding and encodes, whether the relation g is satisfied in the world x_W for the objects $I(o_0), \ldots, I(o_{n-1})$.

Functions in first-order logic are object-valued maps

$$v|_{x_W} \; : \; \mathcal{U}^r \to \mathcal{U} \, .$$

While relations are represented by their coordinate encodings, functions are represented by directed basis encodings

$$\beta^{v|_{x_W}} \left[O_v, O_{[n]} \right]$$

where to each object tuple $I(o_0), \ldots, I(o_{n-1})$ the vector $\beta^{v|_{x_W}} \left[O_v, O_{[n]} = o_{[n]} \right]$ is the one-hot encoding of the object, that is

$$\beta^{v|_{x_W}} \left[O_v, O_{[n]} = o_{[n]} \right] = \epsilon_{v|_{x_W}[I(o_0), \ldots, I(o_{n-1})]} \left[O_v \right] \, .$$

1.1.3 Worlds

Worlds in First-Order Logics contain varying numbers of objects. We use domain variables O to enumerate the sets of objects.

Worlds are represented by three sets of tensors:

- One-hot encodings of constants
- Coordinate encodings for each predicate
- Basis encoding of each function

1.1.4 Two Tensor Structures

In comparison with propositional logic, first-order logic bears two natural tensor structures.

- **Semantic structure:** As in propositional logic, we enumerate possible worlds by a collection of variables X. This is a representation of the model-theoretic semantics, where subsets of worlds are represented by boolean tensors.
- **Substitution structure:** Different to propositional logic, a formula can have variables and the evaluation of a formula on a world is represented by its grounding tensor. Given a world which contains objects in the domain $\mathcal{U}$, we can substitute variables by objects in the domain. In this way, each predicate with r variables is represented in that world as a boolean tensor of order r.

While the semantic structure appears already in factored representations of systems, the substitution structure arises in more generality when treating structured representations of systems [**?**].

1.2 Basis calculus for substitution structure

We here investigate the structure of worlds in first-order logics and how they can be represented by tensors. As we already argued, groundings of sentences can be aligned in boolean tensors, to be called grounding tensors. We here investigate sparse representation schemes for this substitution structure. In particular we apply basis calculus to find sparse decompositions of the grounding tensors

The semantic structure, an attempt to represent in tensors multiple worlds at once, is to be investigated in Chapter **??**.

1.3 Substitution Structure

We in this section investigate the structure of terms and formulas in a single first-order logic world, which we for now index by x_W. As we have derived in Chapter **??** for propositional logic, we develop efficient tensor network representations of the representing tensors based on the corresponding syntax.

1.3.1 Grounding Tensors

Given a first-order logic world x_W, arbitrary formulas are interpreted in terms of the satisfactions of their groundings. We define their semantic first, and then relate their syntactical decomposition to tensor networks, similar to our approach to propositional logic in Chapter **??**.

Definition 1.1 (Grounding of a first-order formula given a world) Given a specific world x_W, with a domain $\mathcal{U}$ enumerated by $[r]$, the grounding of a formula q with variables O_q is the tensor

$$q|_{x_W}\left[O_q\right] : \bigtimes_{l\in[O_q]} [r] \to \{0,1\}\,.$$

Each coordinate represents thereby the boolean, whether the substitution of the variables in the formula is satisfied given a world x_W, that is

$$q|_{x_W}\left[O_q = o_q\right] = 1$$

if and only if the substitution of q with the variables O_q replaced by the objects $I(o_l)$ is satisfied on the world x_W.

We in the following investigate how these tensors are represented by tensor networks involving the world defining tensors.

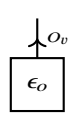
1.3.2 Basis encodings of terms

Terms are object valued functions, which r_v different arguments are also object valued and called variables. We represent terms by the corresponding basis encodings

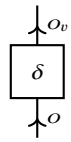
$$\beta^v\left[O_v, O_{[r_v]}\right]$$

Basic blocks:

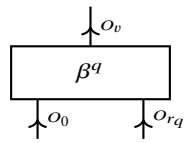
- Constant



- Variable

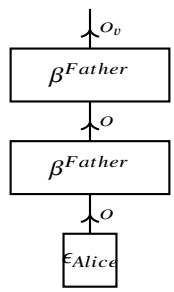


- Function of terms



Contractions of those terms along directed acyclic graphs with single root nodes represent more complex terms. As such, basis encodings of functions can be contracted with constants, variables or other functions to build up basis encodings of more complex terms.

Example: Father of the father of alice is represented by the contraction

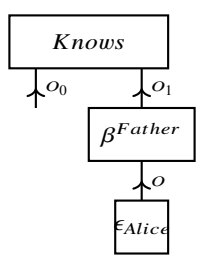


We notice, that the basis encoding of each term is decomposed into a network of world defining tensors, namely constant and function defining tensors.

1.3.3 Atomic sentences

Atomic sentences are contractions of coordinate encodings of predicates with basis encodings of terms.

Example: Knowing the father of alice



Complex sentences are built by

- Quantifier free sentences are contractions of basis encodings to atomic sentences with basis encodings of the connectives.
- Quantifiers act as non-linear transforms of tensors.

1.3.4 Formula Synthesis by Connectives

In order to have a sound semantic, the grounding of first-order logic formulas is determined by the syntax of the formula, i.e. a decomposition of the formula into connectives and quantifiers acting on atomic formulas.

Quantifier-free formulas are connectives acting on atomic formulas. We can describe them as in the case of propositional logic in the β-formalism. While the atomic formulas where delta tensors copying states, they are more involved here.

Theorem 1.1 *For any connective $\circ$ and formulas q_1 and q_2 we have*

$$(q_1 \circ q_2)|_{x_W} \left[O_{q_1} \cup O_{q_2}\right] \tag{1.1}$$

$$= \left\langle \beta^{q_1|_{x_W}} \left[Y_{q_1}, O_{q_1}\right], \beta^{q_2|_{x_W}} \left[Y_{q_2}, O_{q_2}\right], \beta^{\circ} \left[Y_{q_1 \circ q_2}, Y_{q_1}, Y_{q_2}\right], \epsilon_1 \left[Y_{q_1 \circ q_2}\right] \right\rangle_{\left[O_{[n]}\right]} . \tag{1.2}$$

Proof. This directly follows from Thm. **??**. □

Here, variables can be shared by the connected formulas, therefore the variables in the combined formula are unions of the possible not disjoint variables of the connected formulas.

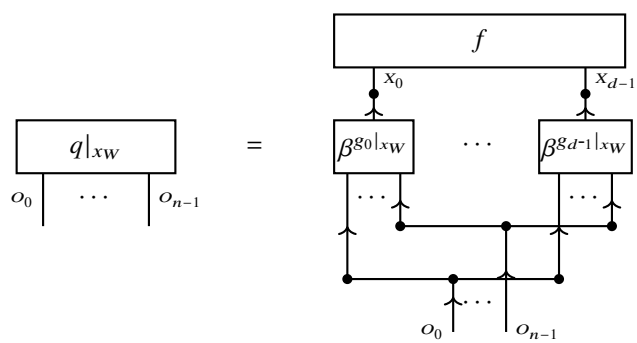
When interpreting the head variables of relational encoded atomic formulas as the atoms of a propositional theory, we find a propositional formula f associated with any decomposable first-order logic formula.

Definition 1.2 Given a formula q in first-order logic, we say that a propositional formula $f\left[X_{[d]}\right]$ is the propositional equivalent to q given atomic formulas g_k in first-order logic, when for any world x_W we have

$$q|_{x_W}\left[O_q\right] = \left\langle \{\beta^{g_k|_{x_W}}\left[X_k, O_{g_k}\right] : k \in [d]\} \cup \{f\left[X_{[d]}\right]\}\right\rangle_{\left[O_q\right]} .$$

We here denote the head variables of the basis encoding to $\beta^{g_k|_{x_W}}$ by X_k to highlight their interpretation as propositional atoms.

We depict the relation of a grounding tensor to a propositional formula as:



1.3.5 Quantifiers

Existential and universal quantifiers appear in generic first-order logic and are besides substitutions further means to reduce the number of variables in a formula.

The semantics of existential quantification consists in a formula being true, if at least one state of the quantified variable is true, as we define next.

Definition 1.3 Given a grounding tensor

$$q|_{x_W}\left[O_0, \ldots, O_{n-1}\right]$$

the existential and universal quantification with respect to the first variable are the tensors

$$\left(\exists_{o_0} q\right)|_{x_W}\left[O_1, \ldots, O_{n-1}\right] \quad \text{and} \quad \left(\forall_{o_0} q\right)|_{x_W}\left[O_1, \ldots, O_{n-1}\right]$$

with coordinates as follows. For an assignment $o_1, \ldots, o$ to the non-quantified variables we have

$$\left(\exists_{o_0} q\right)|_{x_W}\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = 1$$

if and only if there is an assignment $o_0 \in [r_0]$ such that

$$q|_{x_W}\left[O_0 = o_0, O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = 1\,.$$

Conversely, we have for the universal quantification that

$$\left(\forall_{o_0} q\right)|_{x_W}\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = 1$$

if and only if for any assignment $o_0 \in [r_0]$ we have

$$q|_{x_W}\left[O_0 = o_0, O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = 1\,.$$

Let us now show, that existential and universal quantification are coordinatewise transforms (see Def. **??**) of contracted grounding tensors. To this end, let us introduce the greater-z indicator $\mathbb{I}_{>z}$, where $z \in \mathbb{R}$, as the function

$$\mathbb{I}_{>z} : \mathbb{R} \to \{0, 1\} \quad , \quad \mathbb{I}_{>z}\left(x\right) = \begin{cases} 1 & \text{if} \quad x > z \\ 0 & \text{else} \end{cases}\,.$$

Theorem 1.2 *For any formula q with variables $O_{[n]}$ we have*

$$(\exists o_0 q)|_{x_W}\left[O_1, \ldots, O_{n-1}\right] = \mathbb{I}_{>0}\left(\left\langle q|_{x_W}\right\rangle_{[O_1,\ldots,O_{n-1}]}\right)\left[O_1, \ldots, O_{n-1}\right]$$

and

$$(\forall o_0 q)|_{x_W}\left[O_1, \ldots, O_{n-1}\right] = \mathbb{I}_{>r-1}\left(\left\langle q|_{x_W}\right\rangle_{[O_1,\ldots,O_{n-1}]}\right)\left[O_1, \ldots, O_{n-1}\right]$$

Proof. We proof the claimed equalities to arbitrary slices of the remaining variables, which amount to arbitrary substitutions of the formulas. For any indices $o_1 \in [r_1], \ldots, o_{n-1} \in [r_{n-1}]$ We notice that

$$\begin{aligned}\left\langle q|_{x_W}\right\rangle_{[O_1=o_1,\ldots,O_{n-1}=o_{n-1}]} &= \sum_{o_0 \in [r_0]} q|_{x_W}\left[O_0 = o_0, \ldots, O_{n-1} = o_{n-1}\right] \\ &= \left|o_0 \in [r_0] \;:\; q|_{x_W}\left[O_0 = o_0, \ldots, O_{n-1} = o_{n-1}\right] = 1\right|\,.\end{aligned}$$

We can thus understand the contracted grounding tensor as storing in its coordinates the count of the coordinate extensions to the zeroth variable, such that the grounding tensor is satisfied. This is analogous to our interpretation of contracted propositional formulas as world counts. From this it is obvious, that the existential quantification is satisfied, if the count is different from zero, which is captured by the coordinatewise transform with $\mathbb{I}_{>0}$. We therefore arrive at

$$\left(\exists_{o_0} q\right)|_{x_W}\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = \mathbb{I}_{>0}\left(\left\langle q|_{x_W}\right\rangle_{[O_1,\ldots,O_{n-1}]}\right)\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right]\,.$$

The first claim follows, since the assignment to the non-quantified variables was arbitrary. The universal quantification is satisfied, when all extensions are satisfied, and the count is r. Since r is the maximal count, this is captured by the coordinatewise transform with $\mathbb{I}_{>r-1}$ and we get

$$(\forall o_0 q)|_{x_W}\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right] = \mathbb{I}_{>r-1}\left(\left\langle q|_{x_W}\right\rangle_{[O_1,\ldots,O_{n-1}]}\right)\left[O_1 = o_1, \ldots, O_{n-1} = o_{n-1}\right]\,.$$

With the same argument, the second claim is established. □

We can extend this discussion towards more generic counting quantifiers, of which the existential and the universal quantifier are extreme cases. One can define quantifiers by demanding that at least $z \in \mathbb{N}$ compatible groundings are

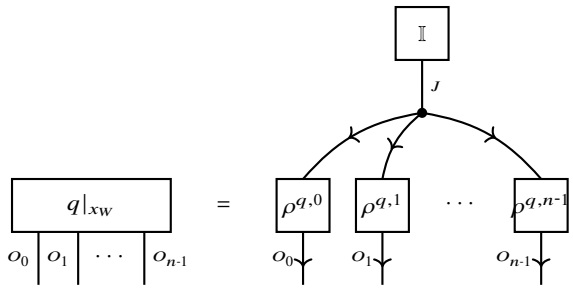


Fig. 1.1 Basis CP Decomposition of the grounding of q, following the scheme of Thm. **??**. Instead of direct storage of the grounding tensor $q|_{x_W}$, the non-zero coordinates are enumerated by a variable J and the corresponding coordinates stored in leg-matrices $\rho^{q,l}$.

satisfied, and show that they amount to coordinatewise transforms with $\mathbb{I}_{>z}$. What is more, quantifiers demanding that at most $z \in \mathbb{N}$ are satisfied would be representable by transforms with an analogously defined function $\mathbb{I}_{\leq z}$. Such customized quantifiers appear for example in the OWL 2 standard of description logics (see [**?**] and Sect. **??**).

As will be discussed in Chapter **??**, any coordinatewise transform can be performed by a contraction of a basis encoding of the tensor with a head vector prepared by the transform function (see Thm. **??**). In the case here, a direct implementation would require a dimension of these head variables by r, which can be infeasible when having large object sets.

To summarize, let us assume a formula is in its prenex normal form, that is a collection of quantifiers are acting on a quantifier free part. We can represent its grounding tensor in three steps:

- Instantiations of the atom groundings with the assigned variables, as contractions of the basis encoding of the world tensor with atom selecting tensors.
- Propositional formula acting on the head variables of the predicate instantiations, representing the connectives combining the formula.
- Quantifiers as a composition of contractions closing the quantified variable and coordinatewise transforms with the respective greater-than indicators.

1.3.6 Storage in Basis CP Decomposition

In many situations, grounding cores are sparse and representations as single tensor cores comes with a drastic overhead. We often encounter sparse grounding tensors, where the number of non-zero coordinates (to be investigated by basis CP ranks in Chapter **??**) satisfies

$$\ell_0\left(q|_{x_W}\right) << r^{|o_q|}\,.$$

In this case, since most coordinates vanish, the basis CP decomposition (see Sect. **??**) enables a representation of the grounding with significantly lower storage demand, see Thm. **??**. This is particularly useful for representing large relational databases, where each object has only a few relations with others, while the majority of possible relations remains unsatisfied. We depict such CP decomposition of a formula grounding in Thm. 1.1.

Most logical syntaxes exploit ℓ_0-sparsity, explicitly storing only known assertions. The interpretation of unspecified assertions depends on the underlying assumptions. Under the Closed World Assumption, for example, all unspecified assertions are assumed to be false.

1.3.7 Example: Relational Databases

A database is understood as a specific first-order logic world, and are operations on such a single world. Queries are described by a formula p, which are asked against a specific world x_W to retrieve the grounding $p|_{x_W}$. The variables

of such formulas are called projection variables. The answer $p|_{x_W}$ of a query is most conveniently represented as a list of solution mappings from the projection variables to objects in the world, such that the query formula is satisfied. Answering a query by solution mappings corresponds with finding the basis CP Decomposition (see Sect. **??**) of $p|_{x_W}$. We can understand these solution mappings as stored in the leg-matrices $\rho^{q,l}$ (see Figure 1.1).

Let us give with the outer join an example of a popular operation to define queries, which efficient execution and storage can be improved based on considerations in the tensor network formalism.

Definition 1.4 (Outer join) Let there be a world x_W and formulas g_ℓ depending on variables $O_{\mathcal{V}^\ell}$, which have grounding tensors by

$$g_\ell|_{x_W}\left[O_v\right] \,:\, \bigtimes_{v\in\mathcal{V}^\ell}\left[r_v\right] \to \{0,1\}\,.$$

Then their (outer) JOIN is defined as the grounding of their conjunctions, as

$$\mathrm{JOIN}\left(g_0,\ldots,g_{p-1}\right)|_{x_W}\left[\bigcup_{\ell\in[p]} O_{\mathcal{V}^\ell}\right] = \left\langle g_\ell|_{x_W}\left[O_{\mathcal{V}^\ell}\right] \,:\, \ell\in[p]\right\rangle_{\left[\bigcup_{l\in[p]} O_{\mathcal{V}^\ell}\right]}\,.$$

We can understand the JOIN of groundings by a factor graph, where each grounding tensor decorates the hyperedge to the node set $\mathcal{V}^\ell$. The projection variable assignment to each formula combined in a JOIN operation provide a basic tensor network format to store the output of the operation. There are thus situations, in which the solution map storage corresponding with a CP Decomposition comes with unnecessary overheads compared with other formats.

We can also understand the JOIN operation as a coordinatewise transform (see Def. **??**) with the product as transform function. To make this connection solid, one would need to extend each joined formula trivially to the variables appearing in other formulas.

The efficiency of evaluating the contraction to a JOIN operation might be improved by understanding it as an Constraint Satisfaction Problem (see Chapter **??**). When applying efficient Message Passing algorithms such as Knowledge Propagation (see Algorithm **??**), the groundings can be sparsified by local constraint propagation operations before turning to more global and more demanding contraction operations. Here the groundings $g_\ell|_{x_W}$ would be used to initialize Knowledge Cores κ^e and sequentially sparsified during the algorithm.

1.4 Knowledge bases

FOL knowledge bases without existential quantifiers are understood as contraction equations. That is, exactly those worlds are models, for which

$$\left\langle\left\{q_\ell\left[O_{[r_{q_\ell}]}\right] \,:\, \ell\in[p]\right\}\right\rangle_{[O_{\mathcal{V}}]} = \mathbb{I}\left[O_{\mathcal{V}}\right]\,.$$

1.4.1 Substitution

Understood as slice restriction of the equation: If the whole tensor is trivial, each slice is trivial. Formal as a constraint network manipulation: Adding a sub-contraction (but with basis vector in addition!).

1.5 Inference

Substitutions are used to lift inference to propositional logics:

- Represent the formulas to be infered with the same computation network of atomic formulas.

- Represent the formulas with an entangled activation network (whereas before they were disentangled, since refering to different atomic sentences).

We understand inference as manipulations in the activation network.

- **Unification**: Substitution of variables to represent multiple sentences by same activation network
- **Propositional inference**: Manipulations in activation network, as for constraint networks

1.5.1 Unification

Unification of atomic sentences: Find a substitution to the variables, such that both sentences are represented by the same computation network.

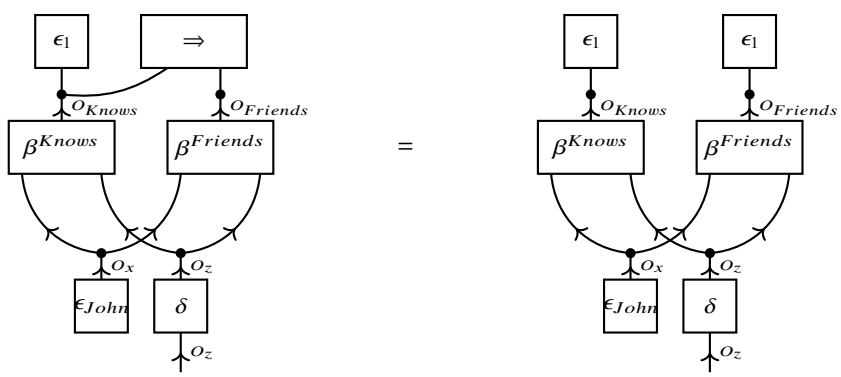
Compressing of complex sentences:

- Find a unification of the atomic sentences in both complex sentences
- Represent the substituted complex sentences by a constraction of the activation networks

1.5.2 Modus ponens

Example 1.1 For example we have

- John knows everyone: Knows(John,z)
- People who know each other are friends: Knows(x,y) ⇒ Friends(x,y)

The unification of Knows(John,z) and Knows(x,y) is given by the substitution $x \mapsto$ John and $y \mapsto z$. We apply the unification on the second rule and get Knows(John,y) ⇒ Friends(John,y).

Now we do modus ponens on the activation network and get Friends(John,z).